Q6: Security Layer — Authentication, Authorization & Hardware Auth

> ***Question:*** *How is authentication and authorization handled? Explain the token implementation and session management. How does the system enforce role-based access control, and how are edge devices or external hardware authenticated?*

Overview

The system uses two completely separate authentication mechanisms side-by-side:

Mechanism	Used By	Location
JWT Bearer Token	Human users (Admin, Manager, Driver)	`Authorization: Bearer <token>` header
API Key	IoT/hardware devices	`X-Hardware-API-Key` header

There is no cookie-based session, no server-side session store for users, and no OAuth2 third-party provider. The system is stateless for human users (the JWT carries all needed identity), with Redis used only for token lifecycle management (refresh + blacklist).

Part 1: Password Security

File: `app/core/security.py`

Hashing

Passwords are hashed with **bcrypt** — a deliberately slow adaptive hash function:

```
@staticmethod
def get_password_hash(password: str) -> str:
    pwd_bytes = password.encode('utf-8')
    salt = bcrypt.gensalt()                        # auto-generates a random 16-byte salt
    hashed = bcrypt.hashpw(pwd_bytes, salt)
    return hashed.decode('utf-8')                  # stored in users.hashed_password
```

`bcrypt.gensalt()` generates a unique random salt per password — two users with the same password produce different hashes. The salt is embedded in the hash string itself, so no separate salt column is needed.

Verification

```
@staticmethod
def verify_password(plain_password: str, hashed_password: str) -> bool:
```

```
    if not plain_password or not hashed_password:
        return False
    return bcrypt.checkpw(
        plain_password.encode('utf-8'),
        hashed_password.encode('utf-8')
    )
```

`bcrypt.checkpw` is constant-time by design — it always does the full work regardless of where strings differ, preventing timing attacks. The early-return `if not plain_password` guard prevents a crash on empty inputs.

Password Strength Validation (Pydantic layer)

Before a password is ever hashed, it is validated in `schemas/schemas.py` via a shared validator applied to `UserCreate` , `SignupRequest` , and `PasswordChangeRequest` :

```
def _validate_password_strength(password: str) -> str:
    if len(password) < 8:
        raise ValueError('Password must be at least 8 characters long')
    if not re.search(r'[A-Z]', password):
        raise ValueError('Password must contain at least one uppercase letter')
    if not re.search(r'[a-z]', password):
        raise ValueError('Password must contain at least one lowercase letter')
    if not re.search(r'\d', password):
        raise ValueError('Password must contain at least one digit')
    if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
        raise ValueError('Password must contain at least one special character')
    return password
```

Weak passwords are rejected with `HTTP 422` before reaching the database layer.

Part 2: JWT Access Tokens

File: `app/core/security.py`

Token Creation

```
@staticmethod
def create_access_token(subject, role=None, expires_delta=None) -> str:
    expire = datetime.now(timezone.utc) + (expires_delta or timedelta(minutes=60))

    to_encode = {
        "exp": expire,       # expiry timestamp (standard JWT claim)
        "sub": str(subject), # subject = user email
    }
    if role:
        to_encode["role"] = str(role)  # custom claim: ADMIN / MANAGER / DRIVER

    return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)
```

JWT payload structure:

```
{
  "exp": 1713293400,
```

```
  "sub": "omar@garage.sa",
  "role": "MANAGER"
}
```

Key design decisions:

- `sub` stores the user's **email** (not a numeric ID) — this is the lookup key in the database
- `role` is embedded in the token payload so role checks need no extra DB round-trip
- Algorithm: **HS256** (HMAC-SHA256) — symmetric signing with `settings.SECRET_KEY`
- Default expiry: **60 minutes** (configurable via `ACCESS_TOKEN_EXPIRE_MINUTES` in `.env`)
- Timezone-aware expiry using `datetime.now(timezone.utc)` — no naive datetime bugs

Token Decoding (Dependency Layer)

File: `app/api/deps.py`

```
async def get_current_user(token, db) -> User:
    try:
        payload = jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM])
        username: str = payload.get("sub")
        if username is None:
            raise credentials_exception
        token_data = TokenData(email=username)
    except JWTError:
        raise credentials_exception   # HTTP 401

    user = await db.execute(select(User).where(User.email == token_data.email))
    if user is None:
        raise credentials_exception   # HTTP 401
    return user
```

`jwt.decode` from `python-jose` validates:

- The signature (using `SECRET_KEY`)
- The `exp` claim (raises `ExpiredSignatureError` if past)
- The algorithm matches

Any failure raises `HTTP 401 Unauthorized` with `WWW-Authenticate: Bearer` header.

Part 3: Refresh Tokens

File: `app/api/v1/auth.py`

Access tokens are short-lived (60 min). Refresh tokens allow clients to get a new access token without re-entering credentials.

Creation

```
@staticmethod
def create_refresh_token() -> str:
    return secrets.token_urlsafe(48)
```

The refresh token is a **64-character URL-safe random string** — completely opaque (no JWT claims, no structure). It contains 48 bytes of cryptographic randomness, making brute-force guessing computationally infeasible.

Storage

Stored in **Redis** (not in the database):

```
await redis.setex(
    f"refresh:{refresh_token}",       # key
    timedelta(days=7),                # TTL — auto-expires after 7 days
    user.email                        # value — maps token → user identity
)
```

The refresh token is the Redis key; the value is the user's email. TTL is set to **7 days** (`REFRESH_TOKEN_EXPIRE_DAYS`).

Exchange

```
@router.post("/refresh", response_model=Token)
async def refresh_access_token(body: RefreshRequest, db):
    email = await redis.get(f"refresh:{body.refresh_token}")
    if not email:
        raise HTTPException(401, "Invalid or expired refresh token")

    user = SELECT User WHERE email = email AND is_active = True
    if not user:
        raise HTTPException(401, "User not found or inactive")

    new_access_token = security.create_access_token(subject=user.email, role=user.role)
    return Token(access_token=new_access_token, token_type="bearer")
```

If Redis is unavailable, `_safe_redis_get` returns `None` gracefully — the client simply cannot refresh and must log in again. The API does not crash.

Token Blacklisting (Password Change)

When a user changes their password, the current access token is revoked:

```
@router.post("/change-password")
async def change_password(request, body, db, current_user):
    # 1. Verify old password
    if not security.verify_password(body.current_password, current_user.hashed_password):
        raise HTTPException(400, "Current password is incorrect")

    # 2. Hash and store new password
    current_user.hashed_password = security.get_password_hash(body.new_password)
    await db.commit()

    # 3. Blacklist the current token in Redis
    token = request.headers["Authorization"][7:]  # strip "Bearer "
    await redis.setex(
        f"blacklist:{token}",
        timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES),  # TTL = remaining token lifetime
        "revoked"
    )
```

The blacklist key expires at the same time the original token would have expired anyway — no unbounded Redis growth.

Redis Graceful Degradation

All Redis calls are wrapped in `_safe_redis_setex` / `_safe_redis_get` :

```
async def _safe_redis_setex(key, ttl, value):
    try:
        redis = _get_redis()
        await redis.setex(key, ttl, value)
        await redis.aclose()
    except Exception as e:
        logger.warning(f"Redis unavailable — skipping setex: {e}")
```

If Redis is down:

- Login still works (refresh token not stored, but access token works for 60 min)
- Refresh endpoint returns 401 (no token to look up)
- Password change still updates the password, but the old token is not blacklisted

The system degrades gracefully rather than failing completely.

Part 4: Role-Based Access Control (RBAC)

File: `app/api/deps.py`

The Three-Layer Dependency Chain

```
# Layer 1 — Decode JWT, fetch user from DB
async def get_current_user(token, db) -> User:
    payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    email = payload.get("sub")
    user = SELECT User WHERE email = email
    return user

# Layer 2 — Verify account is active
async def get_current_active_user(current_user) -> User:
    if not current_user.is_active:
        raise HTTPException(400, "Inactive user")
    return current_user

# Layer 3 — Enforce role
def get_current_user_with_role(*required_roles: UserRole):
    async def role_checker(current_user) -> User:
        if current_user.role not in required_roles:
            raise HTTPException(403, "Not enough permissions")
        return current_user
    return role_checker
```

`get_current_user_with_role` is a **factory function** — it returns a new async dependency function for each set of allowed roles. This allows multi-role expressions like `get_current_user_with_role(UserRole.MANAGER, UserRole.ADMIN)` .

Router-Level vs Endpoint-Level Enforcement

The system uses **router-level** dependencies wherever every endpoint in a file shares the same role requirement:

```
# admin.py — every endpoint requires ADMIN
router = APIRouter(
    dependencies=[Depends(get_current_user_with_role(UserRole.ADMIN))]
)

# driver.py — every endpoint requires DRIVER
router = APIRouter(
    dependencies=[Depends(get_current_user_with_role(UserRole.DRIVER))]
)

# manager_ops.py — every endpoint requires MANAGER or ADMIN
router = APIRouter(
    dependencies=[Depends(get_current_user_with_role(UserRole.MANAGER, UserRole.ADMIN))]
)
```

For endpoints that need the user object (to log actions or verify ownership), the dependency is also declared at endpoint level to inject the user:

```
@router.post("/signup")
async def signup(
    ...,
    current_user: User = Depends(get_current_user_with_role(UserRole.ADMIN))
):
```

FastAPI deduplicates dependencies — the role check is not executed twice even if declared at both levels.

Pre-built Convenience Shortcuts

```
get_current_admin_user   = Depends(get_current_user_with_role(UserRole.ADMIN))
get_current_manager_user = Depends(get_current_user_with_role(UserRole.MANAGER))
get_current_driver_user  = Depends(get_current_user_with_role(UserRole.DRIVER))
```

These are module-level `Depends(...)` instances used directly in endpoint signatures.

Role Enforcement Matrix

Router File	Prefix	Roles Allowed
`auth.py`	`/auth/login` , `/auth/refresh`	None (public)
`auth.py`	`/auth/me` , `/auth/change-password`	Any active user
`auth.py`	`/auth/signup`	ADMIN only
`admin.py`	`/admin/**`	ADMIN only
`users.py`	`/admin/users/**`	ADMIN only
`drivers.py`	`/admin/drivers/**`	ADMIN only

Router File	Prefix	Roles Allowed
`vehicles.py`	`/admin/vehicles/**`	ADMIN only
`routes.py`	`/admin/routes/**`	ADMIN only
`tickets.py`	`/admin/tickets/**`	ADMIN only
`manager_ops.py`	`/manager/tracking/**` , `/manager/reroute/**`	MANAGER or ADMIN
`manager.py`	`/manager/maintenance/**`	MANAGER or ADMIN
`driver.py`	`/drivers/me/**`	DRIVER only
`hardware.py`	`/hardware/**`	API Key (no role)
`websocket.py`	`/ws`	Any valid JWT role

Ownership Enforcement (Driver endpoints)

For driver endpoints, role alone is not enough — a driver must only see their own data. Ownership is enforced explicitly inside each handler:

```
@router.get("/me/trips/{trip_id}")
async def get_my_trip_details(trip_id, db, current_user):
    driver = SELECT Driver WHERE user_id = current_user.id
    trip = await db.get(Trip, trip_id)

    if not trip or trip.driver_id != driver.id:
        raise HTTPException(404, "Trip not found or not assigned to you")
```

Returning `404` (not `403`) on ownership failure is intentional — it avoids confirming to an attacker that the resource exists.

Part 5: Hardware Device Authentication

File: `app/api/v1/hardware.py`

API Key Mechanism

Hardware devices (GPS units, cameras, IoT sensors, ANPR cameras) use a completely separate auth mechanism — no JWT, no user account, no session.

```
async def verify_hardware_api_key(
    x_hardware_api_key: Optional[str] = Header(None)
):
    key = x_hardware_api_key or ""
    if not secrets.compare_digest(key, settings.HARDWARE_API_KEY):
        raise HTTPException(
            status_code=403,
            detail="Invalid or missing hardware API key"
```

```
        )

router = APIRouter(dependencies=[Depends(verify_hardware_api_key)])
```

`secrets.compare_digest` — Timing Attack Prevention

Standard string comparison `==` short-circuits on the first differing character. An attacker sending many requests with slightly different keys can measure response times to determine how many leading characters of their guess are correct.

`secrets.compare_digest` always compares the full strings character-by-character, taking the same time regardless of where they differ. This makes timing-based key discovery computationally infeasible.

```
# Vulnerable to timing attack:
if key == settings.HARDWARE_API_KEY: ...

# Constant-time — used in this codebase:
if not secrets.compare_digest(key, settings.HARDWARE_API_KEY): ...
```

Key Configuration

The API key is stored exclusively in `.env` and loaded via pydantic-settings:

```
class Settings(BaseSettings):
    HARDWARE_API_KEY: str  # No default — must be set explicitly
```

No default value is defined — the application will not start without this key being set in the environment.

Header vs Query Param

The key is passed in the `X-Hardware-API-Key` request header rather than a query parameter. Query parameters appear in:

- Server access logs
- Browser history (if tested via browser)
- Proxy/CDN logs
- Nginx/Uvicorn access logs

Headers do not appear in access logs by default, keeping the key out of log files.

Why No JWT for Hardware?

Concern	JWT	API Key
Stateless validation	Yes (decode locally)	Yes (compare locally)
Expiry	Yes (60 min, needs refresh)	No (permanent until rotated)
Role encoding	Yes	Not needed
Overhead	Higher (decode + DB lookup)	Minimal (one string compare)
Suitable for 120 req/min	Borderline	Yes

Hardware devices POST GPS data up to 120 times per minute. JWT validation requires decoding the token and doing a database lookup on every request. An API key requires only one constant-time string comparison — much lower overhead for high-frequency telemetry.

Part 6: Input Sanitization (XSS Prevention)

File: `app/middleware/sanitization.py`

Before any authentication runs, the request body is sanitized at the ASGI level:

```
sanitized = bleach.clean(value, tags=[], attributes={}, strip=True)
```

`bleach.clean` with `tags=[]` strips all HTML tags from every string field in every JSON body. This prevents XSS payloads (like `<script>alert(1)</script>`) from being stored in the database and later rendered to other users.

This runs **before** Pydantic validation and **before** route handlers — it is the first layer of input defense.

Part 7: Rate Limiting

File: `app/core/rate_limit.py`

```
limiter = Limiter(key_func=get_remote_address, default_limits=["100/minute"])
```

Rate limiting is the last line of defense against brute-force and credential-stuffing:

Endpoint	Limit	Reason
`POST /auth/login`	5/minute	Prevents password brute-force
`POST /hardware/gps`	120/minute	Expected device frequency
`POST /hardware/iot`	60/minute	Expected device frequency
`POST /hardware/anpr`	60/minute	Expected device frequency
`POST /hardware/camera`	60/minute	Expected device frequency
All others	100/minute	Global default

The login endpoint at 5/minute means an attacker can attempt at most 5 passwords per minute per IP address before receiving `HTTP 429 Too Many Requests`.

Part 8: Audit Logging

File: `app/services/audit_service.py`

Every sensitive action writes an audit record:

```
await log_action(db, user_id, "LOGIN", "User", user.id, ip_address=ip)
await log_action(db, user_id, "SIGNUP", "User", new_user.id, ip_address=ip)
await log_action(db, user_id, "CHANGE_PASSWORD", "User", user.id, ip_address=ip)
await log_action(db, user_id, "CREATE", "User", db_user.id, ip_address=ip)
await log_action(db, user_id, "UPDATE", "User", db_user.id, ip_address=ip)
await log_action(db, user_id, "DELETE", "User", db_user.id, ip_address=ip)
```

Each record stores: `user_id` , `action` , `entity_type` , `entity_id` , `old_values` (JSON), `new_values` (JSON), `ip_address` , `created_at` .

The IP address is captured from `request.client.host` , creating a full trail of who did what, from where, and when.

Security Layers in Order (Full Request Path)

```
Incoming request
      |
      v
1. SanitizationMiddleware
   → bleach.clean() all JSON string fields (XSS prevention)
      |
      v
2. IdempotencyMiddleware
   → Redis SETNX check (duplicate mutation prevention)
      |
      v
3. CORSMiddleware
   → Reject unknown origins
      |
      v
4. slowapi RateLimiter
   → Enforce per-IP request caps
      |
      v
5a. [Human routes] JWT verification
   → Decode token, verify signature + expiry
   → DB lookup: SELECT user WHERE email = sub
   → Check is_active = True
   → Check role IN (allowed_roles)
      |
   5b. [Hardware routes] API Key verification
   → secrets.compare_digest(header, env_key)
      |
      v
6. Pydantic schema validation
   → Type coercion, field constraints, password strength
      |
      v
7. Endpoint handler
   → Business logic
   → Audit log write
   → db.commit()
```

Summary

Security Concern	Mechanism
Password storage	bcrypt (adaptive, salted)
Password policy	Pydantic validator (length, upper, lower, digit, special)
Access tokens	JWT HS256, 60-min expiry, role embedded in payload
Refresh tokens	Opaque 64-char random string, stored in Redis with 7-day TTL
Token revocation	Redis blacklist on password change (TTL = remaining token life)
Role enforcement	FastAPI `APIRouter(dependencies=[...])` — router-level, not per-endpoint
Ownership checks	Explicit `driver.id == trip.driver_id` guards in driver endpoints
Hardware auth	API key in header, constant-time compare, no JWT overhead
Timing attack prevention	`secrets.compare_digest` for API key, `bcrypt.checkpw` for passwords
XSS prevention	`bleach.clean` ASGI middleware before any handler runs
Brute-force prevention	slowapi: 5 req/min on `/login` , 100 req/min global
Audit trail	`AuditLog` table: user, action, entity, old/new values, IP address
Redis failure	Graceful degradation — auth still works, refresh/blacklist silently skipped

---

Generated: 2026-04-16